

Lexical Analysis (Scanning)

◆ Basic Concepts & Regular Expressions

- What does a Lexical Analyzer do?
- How does it Work?
- Formalizing Token Definition & Recognition

◆ LEX - A Lexical Analyzer Generator

The Role of the Lexical Analyzer

- ◆ **Lexical Analyzer** (词法分析器) is the first phase of a compiler, it reads the source program character by character to produce tokens for syntax analysis.
- ◆ Normally a lexical analyzer doesn't return a list of tokens at one shot, it returns a token when the parser asks a token from it.
- ◆ **Lexical analyzer = scanning + lexical analysis**

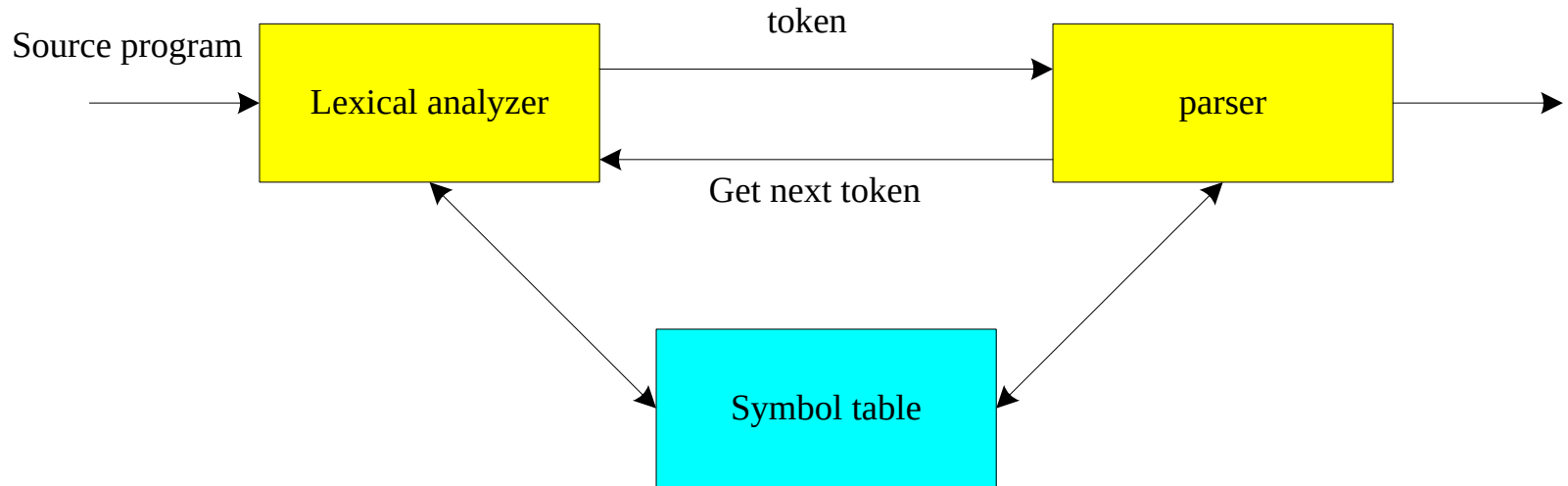


Fig.3.1. interaction of lexical analyzer with parser

What are responsibilities of each box ?

Introducing Basic Terminology

What are Major Terms for Lexical Analysis?

◆TOKEN

- A classification for a common set of strings
- Examples Include <Identifier>, <number>, etc.

◆PATTERN

- The rules which characterize the set of strings for a token
- Recall File and OS Wildcards ([A-Z] *. *)

◆LEXEME

- Actual sequence of characters that matches pattern and is classified by a token
- Identifiers: x, count, name, etc...

Token, pattern, lexemes

- ◆ Since a token can represent more than one lexeme, additional information should be held for that specific lexeme. This additional information is called as the *attribute* (属性) of the token.
- ◆ For simplicity, a token may have a single attribute which holds the required information for that token.
 - ◆ For identifiers, this attribute a pointer to the symbol table, and the symbol table holds the actual attributes for that token.
- ◆ Token type and its attribute uniquely identifies a lexeme.
- ◆ *Regular expressions* (正规表达式) are widely used to specify patterns.

Token, pattern, lexemes

◆ Token is a logical unit in the scanner.

◆ A lexeme is an instance of token.

Token	Sample Lexemes	Informal Description of Pattern
const	const	const
if	if	if
relation	<, <=, =, < >, >, >=	< or <= or = or < > or >= or >
id	pi, <u>count</u> , <u>D2</u>	letter followed by letters and digits
<u>num</u>	<u>3.1416</u> , 0, 6.02E23	any numeric constant
literal	“core dumped”	any characters between “ and “ except “

Classifies
Pattern

Actual values are critical. Info is :

1. Stored in symbol table
2. Returned to parser

Fig.3.2 Examples of tokens.

Attributes for tokens

- ◆ An attribute of the token : any value associated to a token
- ◆ The lexical analyzer collects information about tokens into their associated attributes.
- ◆ The tokens influence parsing decisions; the attributes influence the translation of tokens.
- ◆ Usually a token has only a single attribute—a pointer to the symbol table entry in which the information about the token is kept.
- ◆ Some attributes: (cf. example 3.1)
 - `<id,attr>` where attr is pointer to the symbol table
 - `<assgop,_>` no attribute is needed (if there is only one assignment operator)
 - `<num,val>` where val is the actual value of the number.

Scanning process

- ◆ tokens : the logical units which the scanner generates.
- ◆ Tokens fall into several categories:
 - **Reserved words**: IF, THEN
 - **Identifiers**: ID
 - **operation symbols**: PLUS, MINUS
 - **Numbers**: NUM
 - **Character and string**: STR
 - **Others**: {, }, (,), :, ", ', /, *

Scanning process (cont.)

- ◆ Lexeme (string value): the strings of characters represented by a token.
- ◆ Tokens $\longleftrightarrow$ one lexeme or many lexemes
- ◆ An attribute of the token : any value associated to a token
- ◆ Tokens $\longleftrightarrow$ many attributes
- ◆ The scanner needs to compute at least as many attributes of a token .

Regular Expressions

- ◆ We use regular expressions to describe tokens of a programming language.
- ◆ A regular expression is built up of simpler regular expressions using a set of defining rules.
- ◆ Each regular expression r denotes a language $L(r)$.
- ◆ A language $L(r)$ denoted by a regular expression r is called as a **regular set**.

Regular Expressions (cont.)

- ◆ a regular expression r is defined by the set of strings that it matches.
- ◆ $L(r)$: language generated by the regular expression.
- ◆ **Character set**: be set of the ASCII characters or some subset of it
- ◆ Σ : legal symbols, called alphabet.
- ◆ A regular repression symbols indicates patterns, metacharacters (metasymbols).
- ◆ Backslash and quotes(escape character): turns off the special meaning of a metacharacter.

Rules for Specifying Regular Expressions

fix alphabet Σ

- ◆ ϵ is a regular expression denoting $\{\epsilon\}$
- ◆ If a is in Σ , a is a regular expression that denotes $\{a\}$
- ◆ Let r and s be regular expressions with languages $L(r)$ and $L(s)$.
Then

- $(r) \mid (s)$ is a regular expression $\Rightarrow L(r) \cup L(s)$
- $(r)(s)$ is a regular expression $\Rightarrow L(r) L(s)$
- $(r)^*$ is a regular expression $\Rightarrow (L(r))^*$
- (r) is a regular expression $\Rightarrow L(r)$

All are Left-Associative. Parentheses are dropped as allowed by precedence rules.

Rules for Specifying Regular Expressions (cont.)

Regular expressions over alphabet Σ

<u>Reg. Expr</u>	<u>Language it denotes</u>
ε	$\{\varepsilon\}$
$a \in \Sigma$	$\{a\}$
$(r_1) \mid (r_2)$	$L(r_1) \cup L(r_2)$
$(r_1) (r_2)$	$L(r_1) L(r_2)$
$(r)^*$	$(L(r))^*$
(r)	$L(r)$
$(r)^+ = (r)(r)^*$	$L(r)(L(r))^*$
$(r)? = (r) \mid \varepsilon$	$L(r) \cup \{\varepsilon\}$

Regular Expressions (Ex.)

◆ Ex:

➤ $\Sigma = \{0,1\}$

➤ $0 \mid 1 \Rightarrow \{0,1\}$

➤ $(0 \mid 1)(0 \mid 1) \Rightarrow \{00,01,10,11\}$

➤ $0^* \Rightarrow \{\epsilon, 0, 00, 000, 0000, \dots\}$

➤ $(0 \mid 1)^* \Rightarrow$ all strings with 0 and 1, including the empty string

➤

Specification of tokens

Language Concepts :

A **language, L**, is simply any set of strings over a fixed alphabet.

Alphabet

Languages

$\{0,1\}$

$\{0,10,100,1000,100000\dots\}$

$\{0,1,00,11,000,111,\dots\}$

$\{a,b,c\}$

$\{abc,aabbcc,aaabbbccc,\dots\}$

$\{A, \dots, Z\}$

$\{TEE,FORE,BALL,\dots\}$

$\{FOR,WHILE,GOTO,\dots\}$

$\{A,\dots,Z,a,\dots,z,0,\dots 9,$
 $+, -, \dots, <, >, \dots\}$

$\{ \text{All legal PASCAL progs} \}$

$\{ \text{All grammatically correct English sentences} \}$

Special Languages: $\emptyset$ - EMPTY LANGUAGE

$\{\epsilon\}$ - contains ϵ string only

Terminology of Languages

◆ **Alphabet** : a finite set of symbols (ASCII characters)

◆ **String** :

- Finite sequence of symbols on an alphabet
- Sentence and word are also used in terms of string
- ϵ is the empty string
- $|s|$ is the length of string s .

Terminology of Languages

EXAMPLES AND OTHER CONCEPTS:

Suppose: S is the string `banana`

Prefix : `ban`, `banana`

Suffix : `ana`, `banana`

Substring : `nan`, `ban`, `ana`, `banana`

Subsequence: `bnan`, `nn`

Proper prefix, suffix, or substring *cannot* be all of S

Terminology of Languages (cont.)

◆ Language: sets of strings over some fixed alphabet

- $\emptyset$ the empty set is a language.
- $\{\epsilon\}$ the set containing empty string is a language
- The set of all possible identifiers is a language.

◆ Operators on Strings:

- *Concatenation*: xy represents the concatenation of strings x and y . $s \epsilon = s$ $\epsilon s = s$
- $s^n = s s s \dots s$ (n times) $s^0 = \epsilon$

Operations on Languages

OPERATION	DEFINITION
<i>union</i> of L and M written $L \cup M$	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>concatenation</i> of L and M written LM	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure</i> of L written L^*	$L^* = \bigcup_{i=0}^{\infty} L^i$ L^* denotes “zero or more concatenations of” L
<i>positive closure</i> of L written L^+	$L^+ = \bigcup_{i=1}^{\infty} L^i$ L^+ denotes “one or more concatenations of” L

Fig.3.8 Definitions of operations on languages.

Operations on Languages (Ex3.2)

$$L = \{A, B, C, D\}$$

$$D = \{1, 2, 3\}$$

$$L \cup D = \{A, B, C, D, 1, 2, 3\}$$

$$LD = \{A1, A2, A3, B1, B2, B3, C1, C2, C3, D1, D2, D3\}$$

$$L^2 = \{AA, AB, AC, AD, BA, BB, BC, BD, CA, \dots DD\}$$

$$L^4 = L^2 L^2 = ?? \quad \text{\textcolor{violet}{\{is the set of all four-letter strings\}}}$$

$$L^* = \{ \text{All possible strings of } L \text{ plus } \epsilon \}$$

$$L^+ = L^* - \epsilon \quad \text{\textcolor{brown}{\{is the set of strings beginning with a letter followed by a letter or digit\}}}$$

$$L(L \cup D)^* = ?? \quad \text{\textcolor{violet}{\{is the set of all strings of letters and digits beginning with a letter\}}}$$

Language & Regular Expressions

- ◆ A **Regular Expression** is a Set of Rules / Techniques for Constructing Sequences of Symbols (Strings) From an Alphabet.
- ◆ Let Σ be an Alphabet, r a Regular Expression. Then $L(r)$ is the Language, that is Characterized by the Rules of r .

Regular Expressions (Ex.)

$$L = \{A, B, C, D\} \quad D = \{1, 2, 3\}$$

$$L = A \mid B \mid C \mid D$$

$$L^2 = (A \mid B \mid C \mid D)(A \mid B \mid C \mid D)$$

$$L^* = (A \mid B \mid C \mid D)^*$$

$$L(L \cup D) = (A \mid B \mid C \mid D)((A \mid B \mid C \mid D) \mid (1 \mid 2 \mid 3))$$

AXIOM	DESCRIPTION
$r \mid s = s \mid r$	$\mid$ is commutative
$r \mid (s \mid t) = (r \mid s) \mid t$	$\mid$ is associative
$(r \mid s) t = r (s \mid t)$	concatenation is associative
$r (s \mid t) = r s \mid r t$ $(s \mid t) r = s r \mid t r$	concatenation distributes over $\mid$
$\epsilon r = r$ $r \epsilon = r$	ϵ is the identity element for concatenation
$r^* = (r \mid \epsilon)^*$	relation between $*$ and ϵ
$r^{**} = r^*$	$*$ is idempotent

Fig.3.9 Algebraic Properties of Regular Expressions (P.96)

Regular Expressions (Exercise)

◆ $\Sigma = \{A, \dots, Z, a, \dots, z\}$

All Strings that start with “**tab**” or end with “**bat**”:

$$\text{tab}[A, \dots, Z, a, \dots, z]^* \mid [A, \dots, Z, a, \dots, z]^* \text{bat}$$

◆ $\Sigma = \{A, \dots, Z\}$

All Strings in Which Digits **1,2,3** exist in ascending numerical order:

$$[A, \dots, Z]^* \text{1} [A, \dots, Z]^* \text{2} [A, \dots, Z]^* \text{3} [A, \dots, Z]^*$$

Regular Definitions

- ◆ To write regular expression for some languages can be difficult, because their regular expressions can be quite complex. In those cases, we may use *regular definitions*.
- ◆ We can give names to regular expressions, and we can use these names as symbols to define other regular expressions.

- ◆ A *regular definition* is a sequence of the definitions of the form:

$$\begin{array}{ll} d_1 \rightarrow r_1 & \text{where } d_i \text{ is a distinct name and} \\ d_2 \rightarrow r_2 & r_i \text{ is a regular expression over symbols in} \\ \vdots & \Sigma \cup \{d_1, d_2, \dots, d_{i-1}\} \\ d_n \rightarrow r_n & \end{array}$$

↑
basic symbols

↑
previously defined names

Regular Definitions (cont.)

Regular Definitions: Associate names with Regular Expressions

Example 3.4 : PASCAL IDs

letter $\rightarrow$ A | B | C | ... | Z | a | b | ... | z

digit $\rightarrow$ 0 | 1 | 2 | ... | 9

id $\rightarrow$ letter (letter | digit) *

Shorthand Notation:

✓ “+” : one or more $r^* = r^+ \mid \in \& \quad r^+ = r r^*$

✓ “?” : zero or one $r^? = r \mid \in$

✓ [range] : set range of characters (replaces “|”)

[A-Z] = A | B | C | ... | Z

Example Using Shorthand : PASCAL IDs

id $\rightarrow$ [A-Za-z][A-Za-z0-9]*

Exercise in class

1. Given: $\Sigma = \{a, b, c\}$, describing the set of all strings that contain at most one b with regular expression.

2. Example 3.5:

Given: letter $\rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z$

digit $\rightarrow 0 \mid 1 \mid \dots \mid 9$

Describing identifiers and numbers in Pascal or C with regular expression.

Exercise in class (cont.)

◆ EX: $\Sigma = \{a, b, c\}$

the set of all strings that contain at most one b.

$$(a|c)^*(b|\epsilon)(a|c)^*$$

$$(a|c)^* | (a|c)^*b(a|c)^*$$

the same language may be generated by many different regular expressions.

◆ Ex: Identifiers in Pascal or C

$$\text{letter} \rightarrow A | B | \dots | Z | a | b | \dots | z$$

$$\text{digit} \rightarrow 0 | 1 | \dots | 9$$

$$\text{id} \rightarrow \text{letter} (\text{letter} | \text{digit})^*$$

If we try to write the regular expression representing identifiers without using regular definitions, that regular expression will be complex.

$$(A | \dots | Z | a | \dots | z) ((A | \dots | Z | a | \dots | z) | (0 | \dots | 9))^*$$

Exercise in class (cont.)

Ex: Unsigned numbers in Pascal or C

digit $\rightarrow$ 0 | 1 | ... | 9

digits $\rightarrow$ digit⁺

opt-fraction $\rightarrow$ (. digits) ?

opt-exponent $\rightarrow$ (E (+|-)? digits) ?

unsigned-num $\rightarrow$ digits opt-fraction opt-exponent

Finite Automata (P.113)

- ◆ A *recognizer* for a language is a program that takes a string x , and answers “yes” if x is a sentence of that language, and “no” otherwise.
- ◆ We call the recognizer of the tokens as a *finite automaton*.
- ◆ A finite automaton can be: *deterministic (DFA)* or *non-deterministic (NFA)*
- ◆ This means that we may use a deterministic or non-deterministic automaton as a lexical analyzer.

Finite Automata (cont.)



Finite Automata : A recognizer that takes an input string & determines whether it's a valid sentence of the language

Non-Deterministic : Has more than one alternative action for the same input symbol.

Deterministic : Has at most one action for a given input symbol.

Both types are used to recognize regular expressions.

Finite Automata (cont.)

- ◆ Both deterministic and non-deterministic finite automaton recognize regular sets.
- ◆ Which one?
 - deterministic – faster recognizer, but it may take more space
 - non-deterministic – slower, but it may take less space
 - Deterministic automata are widely used lexical analyzers.
- ◆ First, we define regular expressions for tokens; Then we convert them into a DFA to get a lexical analyzer for our tokens.
 - Algorithm1: Regular Expression $\rightarrow$ NFA $\rightarrow$ DFA (two steps: first to NFA, then to DFA)
 - Algorithm2: Regular Expression $\rightarrow$ DFA (directly convert a regular expression into a DFA)

NFAs & DFAs



Non-Deterministic Finite Automata (NFAs) **easily** represent regular expression, but are somewhat **less precise**.

Deterministic Finite Automata (DFAs) require **more complexity** to represent regular expressions, but offer **more precision**.

We'll review both plus conversion algorithms, i.e.,

NFA $\rightarrow$ DFA and **DFA $\rightarrow$ NFA**

◆ A strong relationship between finite automata and regular expression

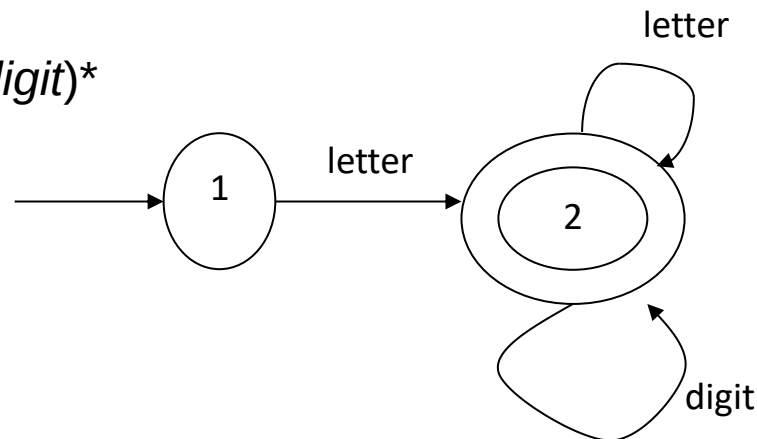
Transition: record a change from one state to another upon a match of the character or characters by which they are labeled.

start state: the recognition process begins drawing an unlabeled arrowed line to it coming “from nowhere”

accepting states: represent the end of the recognition process.

drawing a double-line border around the state in the diagram

- EX.: Identifier $\rightarrow$ letter (letter | digit)*



Non-Deterministic Finite Automata



An **NFA** is a mathematical model that consists of :

- S , a set of **states**
- Σ , the symbols of the **input alphabet**
- *move*, a **transition function**.
 - $move(state, symbol) \rightarrow \text{set of states}$
 - $move : S \times \Sigma \cup \{\epsilon\} \rightarrow \text{Pow}(S)$ (path of word)
- A state, $s_0 \in S$, the **start state**
- $F \subseteq S$, a set of **final** or **accepting states**.

NFA (cont.)

- ◆ ϵ - transitions are allowed in NFAs. In other words, we can move from one state to another one without consuming any symbol.
- ◆ A NFA accepts a string x , if and only if there is a path from the starting state to one of accepting states such that edge labels along this path spell out x .